



طراحی سامانه‌های میکروسرویس

نیم‌سال دوم ۱۴۰۴-۰۵

استاد: دکتر علی ابریشمی - سیدحسین میرحسینی

تمرین اول

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

فهرست مسائل

۱		مسئله ۱
۱		الف
۲		ب
۲		ج
۳		مسئله ۲
۶		مسئله ۳
۷		مسئله ۴
۹		مسئله ۵
۹		شباهت‌های بین دو الگوی Clean Architecture و MVVM
۹		نحوه ادغام و ترکیب دو الگو توسط نویسنده
۹		پروژه‌های مناسب برای استفاده از این الگوی ترکیبی و دلایل آن
۱۱		مسئله ۶

پاسخ مسئله‌ی ۱.

الف

موجودیت یا Entity در واقع همان Critical Business Rules هستند. این کلاس‌ها معمولاً مستقیماً با جدول‌های دیتابیس در ارتباط هستند و دارای ID می‌باشند. همچنین ممکن است شامل منطق و رفتار مرتبط با دامنه‌ی مسئله باشند.

مثال:

```
class User {
    Long id;
    String name;
    String password;
}
```

DTO یا Data Transfer Object شیئی است که فقط برای انتقال داده بین لایه‌های مختلف سیستم استفاده می‌شود. DTO معمولاً منطق خاصی ندارد و فقط شامل اطلاعات موردنیاز است.

مثال:

```
class CreateUserDto {
    String name;
    String password;
}
```

تفاوت اصلی این است که Entity نماینده‌ی داده‌ی اصلی سیستم و مرتبط با دیتابیس است؛ اما DTO فقط برای انتقال داده و کاهش وابستگی یا افزایش امنیت استفاده می‌شود.

ب

اصل CCP بیان می‌کند که کلاس‌هایی که به دلیل مشابهی تغییر می‌کنند، بهتر است در یک پکیج یا ماژول قرار بگیرند. به بیان ساده، اگر چند کلاس معمولاً هم‌زمان تغییر می‌کنند، باید کنار یکدیگر باشند تا مدیریت تغییرات ساده‌تر شود. برای مثال، کلاس‌های مربوط به فاکتور مانند: Invoice, InvoiceService, InvoiceValidator بهتر است در یک پکیج مشترک قرار بگیرند؛ زیرا تغییر قوانین فاکتور احتمالاً باعث تغییر همه‌ی آن‌ها می‌شود. مزیت این اصل:

- کاهش اثر تغییرات روی بخش‌های دیگر
- افزایش نگهداری‌پذیری پروژه
- ساده‌تر شدن توسعه و تست

ج

Decoupling به معنی کاهش وابستگی بین بخش‌های مختلف سیستم است. هرچه اجزای سیستم مستقل‌تر باشند، توسعه و نگهداری ساده‌تر خواهد بود. منظور از سطح Deployment این است که اجزای سیستم چگونه منتشر و اجرا می‌شوند. در یک سیستم کاملاً Decoupled، هر بخش می‌تواند به صورت مستقل Deploy شود. برای مثال در معماری Microservice سرویس کاربران و سرویس پرداخت و سرویس سفارش هرکدام جداگانه اجرا و منتشر می‌شوند. اما در معماری Monolith، معمولاً کل سیستم باید با هم Deploy شود.

مزایای Deployment مستقل:

- سرعت بیشتر در انتشار تغییرات
- کاهش وابستگی بین نسخه‌ها
- امکان توسعه‌ی مستقل توسط تیم‌ها
- کاهش ریسک هنگام انتشار تغییرات

پاسخ مسئله‌ی ۲.

الهام‌گیری Architecture Clean از اصل DIP

اصل وارونگی وابستگی شامل دو بند اساسی است:

۱. ماژول‌های سطح بالا نباید به ماژول‌های سطح پایین وابسته باشند؛ هر دو باید به انتزاع وابسته باشند.
 ۲. انتزاع‌ها نباید وابسته به جزئیات باشند، بلکه جزئیات (پیاپیاده‌سازی‌های واقعی) باید وابسته به انتزاع‌ها باشند.
- در Clean Architecture، نرم‌افزار به لایه‌های متمرکزی تقسیم می‌شود که هسته مرکزی آن شامل بالاترین سطح از سیاست‌ها و منطق تجاری برنامه (Domain و Use Cases) است، و لایه‌های بیرونی شامل ابزارها و مکانیزم‌های سطح پایین (Frameworks, Databases, UI) هستند.
- قانون طلایی این معماری: جهت وابستگی‌های کدهای منبع همیشه باید رو به داخل و به سمت سیاست‌های سطح بالا باشد.
- تناقض معماری و حل آن به کمک وارونگی در عمل یک تناقض پیش می‌آید: یک Use Case سطح بالا (مثلاً RegisterUser) برای انجام کار خود نیاز دارد داده‌ای را ذخیره کند یا ایمیلی بفرستد، که این‌ها عملیاتی سطح پایین و مربوط به ابزارهای بیرونی هستند.
- Clean Architecture این تناقض را دقیقاً با الهام از DIP حل می‌کند: به جای اینکه Use Case مستقیماً به کدهای دیتابیس وابسته شود، یک اینترفیس در داخل لایه خود تعریف می‌کند. لایه دیتابیس در بیرون، این اینترفیس را پیاده‌سازی می‌کند. در نتیجه، وابستگی زمان کامپایل کاملاً وارونه شده و به سمت داخل متمایل می‌شود، در حالی که جریان داده در زمان اجرا به سمت بیرون است.

مشکلات ساختاری که با رعایت این اصل حل می‌شوند

- **Coupling شدید:** بدون DIP، هرگونه تغییر در جزئیات دیتابیس (مثلاً مهاجرت از PostgreSQL به MongoDB) یا تغییر در ابزارهای بیرونی، منطق اصلی کسب‌وکار را خراب می‌کند. DIP هسته سیستم را از نوسانات زیرساخت ایزوله می‌کند.
- **تست‌پذیری ضعیف:** وقتی منطق برنامه به سیستم‌های واقعی متصل باشد، نوشتن Unit Test سریع غیرممکن است. با وابستگی به انتزاع‌ها، می‌توان در محیط تست به راحتی ساختارهای فرضی را تزریق کرد.
- **اختلال در توسعه موازی:** تیم توسعه لایه تجاری می‌تواند بدون منتظر ماندن برای نهایی شدن ساختار دیتابیس یا سرویس‌های شبکه، کدهای خود را به طور مستقل بنویسد و تست کند.

نحوه متصل شدن دو لایه به یکدیگر در زمان اجرا

بر اساس قانون وابستگی، لایه‌های درونی هیچ دیدی به لایه‌های بیرونی ندارند. اما در زمان اجرا، جریان کنترل برای دسترسی به دیتابیس باید از لایه تجاری خارج شود. برای حل این تناقض بدون مخدوش شدن مرزهای معماری، از الگوی Dependency Injection استفاده می‌شود. در این روش، لایه بالا نیازمندی خود را به صورت یک اینترفیس اعلام می‌کند. سپس در بیرونی‌ترین نقطه برنامه یعنی Composition Root (مانند تابع main)، پیاده‌سازی واقعی دیتابیس ساخته شده و به لایه بالا تزریق می‌شود. مراحل کدی این اتصال به شرح زیر است:

تعریف انتزاع و مدل در لایه سطح بالا (Core)

```

1 package usecases
2
3 // Domain Entity -
4 type User struct {
5     ID    string
6     Name  string
7 }
8
9 // UserRepository -
10 type UserRepository interface {
11     GetByID(id string) (*User, error)
12 }

```

پیاده‌سازی مستقل در لایه سطح پایین (Infrastructure)

```

1 package infrastructure
2
3 import (
4     "database/sql"
5     "errors"
6     "usecases"
7 )
8
9 type DBUser struct {
10     ID          string
11     FullName    string
12     CreatedAt   string
13 }
14
15 type SQLUserRepository struct {
16     db *sql.DB
17 }
18
19 func NewSQLUserRepository(db *sql.DB) *SQLUserRepository {
20     return &SQLUserRepository{db: db}
21 }
22
23 func (r *SQLUserRepository) GetByID(id string) (*usecases.User, error) {
24     query := "SELECT * FROM users WHERE id = $1;"
25
26     var dbUser DBUser
27
28     err := r.db.QueryRow(query, id).Scan(&dbUser.ID, &dbUser.FullName, &dbUser.CreatedAt)
29     if err != nil {
30         if errors.Is(err, sql.ErrNoRows) {
31             return nil, nil
32         }
33         return nil, err
34     }
35
36     domainUser := &usecases.User{
37         ID:    dbUser.ID,
38         Name:  dbUser.FullName,
39     }
40
41     return domainUser, nil
42 }

```

تزریق وابستگی در نقطه ورود (Composition Root)

```
1 package main
2
3 import (
4     "database/sql"
5     _ "github.com/lib/pq"
6     "infrastructure"
7     "usecases"
8 )
9
10 func main() {
11     dbConn, err := sql.Open("postgres", "postgres://user:pass@localhost/db")
12     if err != nil {
13         panic(err)
14     }
15     defer dbConn.Close()
16
17     userRepoImpl := infrastructure.NewSQLUserRepository(dbConn)
18
19     userUseCase := usecases.NewUserUseCase(userRepoImpl)
20
21     user, err := userUseCase.Execute("123")
22 }
```

پاسخ مسئله‌ی ۳.

چارچوب‌محور بودن به این معناست که معماری، ساختار پوشه‌ها، و منطق تجاری نرم‌افزار کاملاً بر اساس قوانین، ساختار و امکانات یک فریمورک خاص (مانند Django Spring Laravel, NestJS) شکل بگیرد. در این حالت، ابزارها و فریمورک‌ها در قلب سیستم قرار دارند و کدهای هسته برنامه به شدت به کامپوننت‌های داخلی فریمورک وابسته می‌شوند.

وابستگی شدید به یک فریمورک مشکلات جدی در طول عمر پروژه ایجاد می‌کند:

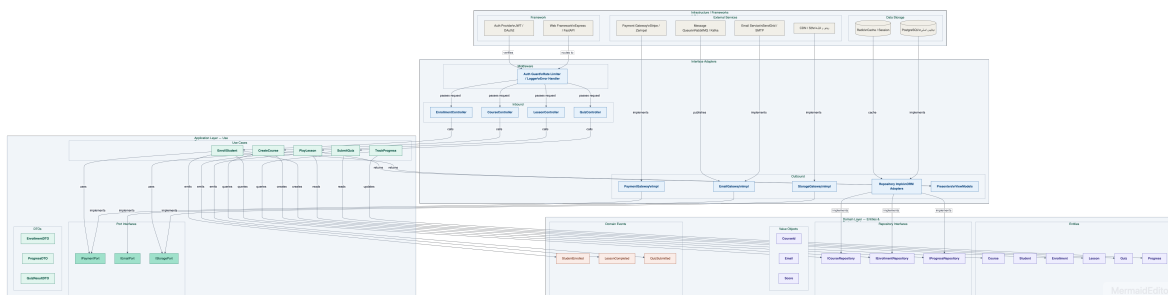
- **وابستگی شدید:** منطق بیزینس مستقیماً کلاس‌های فریمورک را ارث‌بری می‌کند یا از ابزارهای آن استفاده می‌کند. اگر فریمورک تغییر کند یا منسوخ شود، بخش زیادی از سیستم باید بازنویسی شود.
 - **دشواری در تست‌نویسی:** برای نوشتن یک Unit Test ساده روی منطق بیزینس، مجبورید ساختار فریمورک، وب‌سرور یا دیتابیس متصل به آن را شبیه‌سازی یا اجرا کنید که سرعت و دقت تست‌ها را کاهش می‌دهد.
 - **کابوس به‌روزرسانی:** با معرفی نسخه‌های جدید فریمورک، به‌روزرسانی پروژه بسیار زمان‌بر، پرهزینه و خطرناک خواهد بود.
 - **محدودیت در انعطاف‌پذیری:** توسعه‌دهنده مجبور است کدهای خود را با ساختار پیشنهادی فریمورک هماهنگ کند، حتی اگر آن ساختار برای حل مسئله‌ی خاص بیزینس بهینه نباشد.
- Clean Architecture با تغییر جهت وابستگی‌ها و قرار دادن منطق بیزینس در مرکز، این مشکلات را حل می‌کند:

- **قانون وابستگی:**
 - در این معماری، لایه‌ها به صورت دایره‌های متمرکز طراحی می‌شوند. قانون طلایی این است: وابستگی‌ها فقط باید به سمت داخل باشند. یعنی لایه‌های درونی (بیزینس) هیچ اطلاعی از لایه‌های بیرونی (فریمورک، دیتابیس) ندارند. فریمورک صرفاً یک ابزار در بیرونی‌ترین لایه است.
 - **جداسازی منطق تجاری:**
 - Entities: قوانین کلی و مستقل بیزینس را کپسوله‌سازی می‌کنند.
 - Use Cases: منطق اختصاصی هر عملیات سیستم را پیاده‌سازی می‌کنند.
- این دو لایه مرکزی کاملاً با کد خالص بدون وارد کردن پکیج‌ها یا دکوراتورهای فریمورک نوشته می‌شوند.
- **اصل وارونگی وابستگی:**
 - اگر لایه داخلی نیاز به ارتباط با یک ابزار بیرونی (مانند دیتابیس یا سرویس پیام‌رسان) داشته باشد، مستقیماً به آن وصل نمی‌شود؛ بلکه یک اینترفیس در لایه داخلی تعریف می‌کند و فریمورک در لایه بیرونی موظف است آن را پیاده‌سازی کند.

با استفاده از Clean Architecture فریمورک تبدیل به یک جزئیات می‌شود، نه محور سیستم. شما می‌توانید لایه وب یا دیتابیس فریمورک را بدون دست زدن به کدهای اصلی بیزینس تعویض یا به‌روزرسانی کنید.

پاسخ مسئله‌ی ۴.

دیاگرام معماری:



شکل ۱: معماری Clean Architecture سامانه Courseware

لایه Domain — هسته‌ی مرکزی

این لایه هیچ وابستگی به هیچ کتابخانه‌ی خارجی ندارد و خالص‌ترین بخش سیستم است. اجزای آن عبارتند از:

- Entities: Student, Lesson, Course, Progress, Quiz, Enrollment, Score
- Value Objects: CourseId, Email, Score, Duration
- Domain Events: StudentEnrolled, LessonCompleted, QuizSubmitted
- Repository Interfaces: قراردادهایی که لایه‌ی Infrastructure باید پیاده‌سازی کند

لایه Application — UseCases

منطق تجاری اپلیکیشن را کنترل می‌کند و فقط به لایه‌ی Domain وابسته است.

Use Case	مستولیت
EnrollStudent	اعتبارسنجی ثبت‌نام، پردازش پرداخت، ایجاد Enrollment
PlayLesson	بررسی دسترسی، تولید URL امن محتوا، ثبت شروع مشاهده
SubmitQuiz	دریافت پاسخ‌ها، نمره‌دهی خودکار، ذخیره QuizResult
TrackProgress	محاسبه درصد پیشرفت، صدور گواهینامه در صورت تکمیل
CreateCourse	ایجاد دوره توسط مدرس، اعتبارسنجی محتوا

جدول ۱: UseCase های اصلی لایه‌ی Application

این لایه برای ارتباط با سرویس‌های خارجی فقط interface تعریف می‌کند. پیاده‌سازی واقعی در Infrastructure است.

لایه Interface Adapters

داده را بین دنیای خارج و لایه‌ی Application تبدیل می‌کند.

- Controllers: HTTP request را دریافت، اعتبارسنجی، و به DTO تبدیل می‌کنند
- Presenters: خروجی use case را به فرمت JSON مناسب برای کلاینت تبدیل می‌کنند

- **Repository Implementations**: پیاده‌سازی واقعی interface های Domain با استفاده از ORM
- **External Gateways**: پیاده‌سازی Port Interface ها
- **Middleware**: Error Handler ،Logger ،Rate Limiter ،Auth Guard

لایه Drivers & Frameworks — Infrastructure

جزئیات فنی که قابل تعویض هستند و هیچ‌کدام به لایه‌های داخلی وابسته نیستند.

بخش	توضیح	فناوری
دیتابیس اصلی	ذخیره‌سازی دوره‌ها و ثبت‌نام‌ها	PostgreSQL
Cache	کش query ها و session کاربران	Redis
Storage / CDN	ویدیوها و فایل‌های درسی	AWS S3
درگاه پرداخت	پردازش تراکنش‌های مالی	Zarinpal
ارسال ایمیل	تأیید ثبت‌نام و یادآوری‌ها	SMTP
Queue Message	رویدادهای async	RabbitMQ / Kafka
فریم‌ورک وب	HTTP Server	FastAPI

جدول ۲: اجزای لایه‌ی Infrastructure

ماژول‌های اصلی سامانه

مسئولیت	ماژول
ایجاد، ویرایش، و انتشار دوره‌ها توسط مدرس	Course Management
ثبت‌نام دانش‌آموز، پرداخت، مدیریت دسترسی	Enrollment
پخش محتوا، ردیابی پیشرفت، مدیریت نشانک‌ها	Learning Engine
آزمون‌ها، تکالیف، نمره‌دهی، صدور گواهینامه	Assessment
احراز هویت، پروفایل، نقش‌ها	User Management
ایمیل، پوش نوتیفیکیشن، یادآوری ادامه‌ی درس	Notification
گزارش پیشرفت دانش‌آموزان و آمار دوره‌ها	Analytics

جدول ۳: ماژول‌های اصلی سامانه‌ی Courseware

ماژول‌ها مستقیم به هم وابسته نیستند و از طریق Domain Events ارتباط برقرار می‌کنند. وقتی StudentEnrolled منتشر می‌شود، ماژول Notification آن را دریافت و ایمیل می‌فرستد، بدون اینکه ماژول Enrollment چیزی درباره‌ی Notification بداند. این الگو Loose Coupling را تضمین می‌کند و تعویض هر بخش را بدون تأثیر بر بقیه ممکن می‌سازد.

پاسخ مسئله‌ی ۵.

شباهت‌های بین دو الگوی Clean Architecture و MVVM

با توجه به مباحث مطرح شده در متن مقاله و Case Study، شباهت‌های بنیادی و ساختاری زیر بین این دو الگو دیده می‌شود:

- **Separation of Concerns**: هر دو الگو با هدف تفکیک وظایف لایه‌های مختلف سیستم طراحی شده‌اند. الگوی MVVM رابط کاربری را از منطق نمایش جدا می‌کند و Clean Architecture کل نرم‌افزار را به لایه‌های مستقل تقسیم می‌نماید.
- **Testability**: در هر دو ساختار، بستری عالی برای Unit Testing مهیا شده است. در MVVM، لایه ViewModel مستقل از UI تست می‌شود و در Clean Architecture نیز هر لایه به طور جداگانه قابل ارزیابی است.
- **Modularity**: هر دو الگو کدها را به بخش‌های مجزا و مستقل تقسیم می‌کنند که توسعه موازی و مدیریت پروژه را تسهیل می‌کند.
- **پیچیدگی و سربار در پروژه‌های کوچک**: مقاله به صراحت اشاره می‌کند که هر دو الگو برای پروژه‌های ساده و کوچک، به دلیل ایجاد کدهای تکراری زیاد و پیچیدگی اولیه، نوعی زیاده‌روی به شمار می‌روند.

نحوه ادغام و ترکیب دو الگو توسط نویسنده

نویسنده مقاله برای بهره‌گیری از نقاط قوت هر دو ساختار، آن‌ها را به شکل یک معماری هیبریدی منسجم ادغام کرده است. نقطه عطف این ترکیب در لایه واسط قرار دارد. نویسنده ViewModel (از الگوی MVVM) را دقیقاً هم‌تراز و معادل لایه Interface Adapter (از الگوی Clean Architecture) قرار داده است با این کار، وظیفه مدیریت وضعیت نمایش را بر عهده می‌گیرد در حالی که با لایه‌های درونی‌تر برای اجرای منطق تجاری در ارتباط است.

جریان ارتباطی لایه‌ها در لایه‌بندی ترکیبی:

۱. **View (لایه UI)**: بیرونی‌ترین سطح که با کاربر در تعامل است و از طریق فرآیند Data Binding به ویژگی‌های ViewModel متصل می‌شود.
۲. **ViewModel (لایه Presentation)**: نقش واسط را ایفا کرده، ورودی‌های View را گرفته و Use Case متناسب را فراخوانی می‌کند.
۳. **Use Cases (لایه Business Logic)**: حاوی منطق‌های خاص هر قابلیت برنامه (مانند افزودن یا حذف یک Task) است.
۴. **Entities (لایه Core Rules)**: شامل اشیاء پایه و قوانین بنیادی کسب‌وکار است که به هیچ فریمورک خارجی وابستگی ندارد.
۵. **Data Layer**: وظیفه تعامل با پایگاه داده محلی یا API‌های سرور و مدیریت ماندگاری داده‌ها را بر عهده دارد.

پروژه‌های مناسب برای استفاده از این الگوی ترکیبی و دلایل آن

بر اساس تحلیل مزایا و معایب در متن مقاله، این معماری ترکیبی برای پروژه‌هایی با ویژگی‌های زیر بیشترین بازدهی را دارد:

- **پروژه‌های بزرگ و سازمانی:** به دلیل ساختار کاملاً مازولار، این الگو اجازه می‌دهد تیم‌های بزرگ بدون تداخل با یکدیگر روی بخش‌های مختلف پروژه کار کنند.
 - **پروژه‌های با طول عمر بالا و نیازمند نگهداری:** از آنجا که لایه‌ها به وسیله اینترفیس‌ها از هم ایزوله شده‌اند، تغییر در یک لایه (مثلاً تعویض کتابخانه شبکه یا تغییر دیتابیس) تأثیری روی منطق اصلی برنامه نخواهد گذاشت.
 - **پروژه‌های مبتنی بر TDD:** برای برنامه‌هایی که پوشش تست بالایی نیاز دارند، تفکیک دقیق لایه‌ها امکان نوشتن تست‌های واحد پایدار و مستقل را فراهم می‌سازد.
- پیاده‌سازی این ساختار نیازمند تعریف اینترفیس‌های متعدد، کلاس‌های مختلف و تنظیمات اولیه پیچیده‌ای است در پروژه‌های کوچک، زمان و هزینه‌ای که صرف نوشتن این کدهای اضافه می‌شود، توجیه اقتصادی و فنی ندارد.

پاسخ مسئله‌ی ۶.

کاربرد Clean Architecture در طراحی این پروژه

ایده‌ی اصلی Clean Architecture: (۱) قوانین کسب‌وکار باید از جزئیات فناوری جدا باشند، و (۲) جهت وابستگی‌ها فقط رو به داخل، یعنی به سمت Entities و Use Cases است. هیچ کلاسی در لایه‌های داخلی حق ندارد به یک کتابخانه یا فریمورک خارجی اشاره کند.

• **لایه‌ی Entities (Domain):** موجودیت Message، Value Object، ای به نام QueueStatus، portهای Queue، sentinel error، QueueFactory، QueueManager و Logger، و مجموعه‌ای از خطاهای کسب‌وکار به صورت ErrQueueAlreadyExists، ErrQueueNotFound، ErrQueueEmpty، ErrQueueFull، ...). این لایه هیچ import خارجی ندارد.

• **لایه‌ی UseCases:** کلاس QueueUseCase منطق برنامه را مدیریت می‌کند و دو port خروجی به نام‌های IDGenerator و Clock تعریف می‌کند تا حتی به uuid یا time.Now مستقیم وابسته نباشد. این لایه از طریق port ورودی QueueService توسط لایه‌ی Delivery مصرف می‌شود.

• **لایه‌ی Interface Adapters - Repository:** پیاده‌سازی in-memory صف به صورت Ring Buffer (RingBufferQueue) و یک QueueManager امن از نظر هم‌زمانی (با RWMutex). QueueManager از طریق یک QueueFactory تزریق شده، نوع صف زیرین را نمی‌داند؛ یعنی می‌توان بعداً Redis-backed Queue اضافه کرد بدون اینکه این کلاس تغییر کند.

• **لایه‌ی Interface Adapters - Delivery:** بخش HTTP handlerهای مبتنی بر Gin، DTOها، یک router مستقل، و یک تابع متمرکز writeDomainError که خطاهای دامنه را به کد وضعیت HTTP نگاشت می‌کند. تمام این لایه تنها به usecase.QueueService interface وابسته است، نه به نوع concrete.

• **لایه‌ی Service:** TTLCleaner که با یک ticker و context.Context پیام‌های منقضی را پاکسازی می‌کند و قابلیت graceful shutdown دارد. این worker فقط به پورت‌های QueueManager و Logger وابسته است.

• **لایه‌ی Frameworks & Drivers:** Gin، logger.Zap، clock.Real، idgen.UUID، Composition Root (یعنی cmd/server/main.go) به لایه‌های داخلی تزریق می‌شوند. این‌ها فقط در

رعایت اصل DIP: لایه‌ی UseCase port موردنیاز خود را خودش تعریف می‌کند (Clock، IDGenerator) و لایه‌ی بیرونی موظف است آن را پیاده کند.

نتیجه: تعویض uuid با هر تولیدکننده‌ی ID دیگر، یا تعویض Gin با Echo/Fiber/gRPC، تنها یک adapter جدید می‌خواهد و دستکاری در منطق کسب‌وکار لازم نیست.

نکات کلیدی پیاده‌سازی:

• پیاده‌سازی صف به صورت Ring Buffer با پیچیدگی $O(1)$ برای push و pop و بدون allocation اضافه به ازای هر پیام.

• انقضای پیام به دو شیوه‌ی موازی: (۱) Lazy هنگام Pop: پیام‌های منقضی شده در ابتدای صف رد می‌شوند، (۲) Proactive از طریق TTLCleaner که هر یک ثانیه یکبار اجرا می‌شود تا حتی پیام‌هایی که هرگز pop نمی‌شوند نیز حافظه را نگه ندارند.

• قرارداد: TTL=0 به معنی پیام بدون تاریخ انقضاست.

• همه‌ی خطاهای کسب‌وکار به صورت مقدارهای sentinel (errors.New) در پکیج domain تعریف شده‌اند و فقط در یک نقطه‌ی متمرکز (تابع writeDomainError) به کد وضعیت HTTP نگاشت می‌شوند. این یعنی هیچ مفهوم HTTP ای داخل لایه‌های داخلی نشأت نمی‌کند.

- Composition Root در cmd/server/main.go سیگنال‌های SIGINT/SIGTERM را با signal.NotifyContext می‌گیرد، سپس TTLCleaner و http.Server را با مهلت ۵ ثانیه‌ای Shutdown می‌کند.

مسیرهای HTTP پیاده‌سازی شده:

```
GET    /healthz          # liveness probe
POST   /queues           # create queue (capacity required)
GET    /queues           # list queues
GET    /queues/:queue    # status: {id, name, size, capacity}
POST   /queues/:queue/push # enqueue {value, ttl}
POST   /queues/:queue/pop  # dequeue; 204 if empty
```

کتابخانه‌ها و دلیل انتخاب آن‌ها:

- github.com/gin-gonic/gin: HTTP router سبک با middleware منعطف. فقط در لایه‌ی delivery استفاده می‌شود.
- github.com/google/uuid: تولید UUID v4 برای شناسه‌ی صف‌ها و پیام‌ها. در پشت IDGenerator port ایزوله شده.
- go.uber.org/zap: logger ساخت‌یافته و سریع. در پشت domain.Logger port ایزوله شده و فقط در internal/pkg/logger حضور دارد.

اجرای پروژه:

```
cd Baby-Rabbit
go run ./cmd/server          # server listens on :8080
```

تست با Postman: فایل Baby-Rabbit.postman_collection.json شامل تمام request ها به همراه test script های JavaScript است که به صورت خودکار شناسه‌ی صف ساخته شده را در collection variable ذخیره می‌کند. کافی است collection را import کنید و درخواست‌ها را به ترتیب اجرا کنید.

اعتبارسنجی قاعده‌ی وابستگی: می‌توان به صورت تجربی نشان داد که هیچ‌یک از لایه‌های داخلی به فریمورک‌های بیرونی وابسته نیستند:

```
go list -f '{{join .Imports "\n"}}' ./internal/domain
# (empty - pure Go)
```

```
go list -f '{{join .Imports "\n"}}' ./internal/usecase
# Baby-Rabbit/internal/domain
# strings
# time
```

```
go list -f '{{join .Imports "\n"}}' ./internal/service
# Baby-Rabbit/internal/domain
# context
# time
```

به عبارت دیگر، با گرفتن یک snapshot از import graph می‌بینیم که هیچ ارجاعی از domain/usecase/service به zap، uuid، gin یا حتی net/http وجود ندارد.

تحلیل نهایی معماری

اگر معماری به دست آمده را با دوایر هم‌مرکز اصلی معماری Clean مقایسه کنیم، تطابق به صورت زیر است:

- Entities $\Leftrightarrow$ بسته‌ی domain (Message و port‌های QueueManager/Queue).
- Use Cases $\Leftrightarrow$ بسته‌ی usecase (QueueUseCase و port‌های آن).
- Interface Adapters $\Leftrightarrow$ بسته‌های http/delivery و repository.
- Frameworks & Drivers $\Leftrightarrow$ gin، uuid، zap، net/http که فقط در لایه‌ی بیرونی و در Composition Root ظاهر می‌شوند.

قاعده‌ی وابستگی برقرار است: هیچ فایلی در domain و usecase از پکیج‌های gin، uuid یا zap import نمی‌کند و این موضوع را با `go list -deps` می‌توان به صورت آجکتیو تأیید کرد.

نزدیک‌ترین الگو: از منظر الگوها، معماری نهایی به Hexagonal Architecture (که با نام Ports & Adapters نیز شناخته می‌شود) بسیار نزدیک است. دلیل این نزدیکی روشن است: port‌های ورودی (QueueService) و port‌های خروجی (QueueFactory، Queue، QueueManager، Clock، IDGenerator، Logger) به صورت صریح در پکیج‌های داخلی تعریف شده‌اند و هر adapter (Zap، System Clock، UUID، Ring Buffer، Gin HTTP) دقیقاً روی لایه‌ی شش ضلعی قرار دارد. در عمل، Hexagonal و Clean Architecture در همین پروژه به یک‌جا می‌رسند: تفاوت آن‌ها بیشتر در ادبیات (layers در برابر hexagon) است تا در ساختار نهایی کد. اگر بخواهیم دقیق‌تر بگوییم، چون ما در پایین مرز شش ضلعی هیچ driving adapter (مثل CLI یا message consumer) نداشتیم و تنها یک HTTP adapter ورودی داریم، حالت ساده‌شده‌ی Hexagonal است.

مزایای ملموسی که این طراحی به ما داد:

- جایگزینی فناوری در لایه‌ی بیرونی (مثلاً مهاجرت از Gin به Echo یا حتی به gRPC) تنها همان لایه را دست‌خوش تغییر می‌کند.
- جایگزینی Ring Buffer با هر backing store دیگری (مثلاً Redis یا BadgerDB) فقط یک QueueFactory تازه می‌خواهد و یک خط در `main.go`. لایه‌های usecase و domain دست‌نخورده می‌مانند.
- جایگزینی zap با slog یا logr فقط یک adapter تازه برای domain.Logger می‌خواهد.
- منطق برنامه بدون نیاز به HTTP server، uuid یا wall clock قابل تست است و این موضوع در پوشش تست‌های واحد منعکس شده است.